

## Unidad 3. Tipos de Datos Estructurados

### Enumeraciones

Una enumeración es un conjunto de constantes enteras. De forma automática se suelen iniciar en 0 y luego continúan en forma ascendente.

A la enumeración se le puede asignar un nombre, que se comportará como un nuevo tipo de dato que solo podrá contener los valores especificados en la enumeración.

Las declaraciones se pueden hacer dentro o fuera del programa principal. Existen enumeraciones anónimas, esto es que no disponen de un nombre para referirse a la enumeración. Ejemplo:

```
enum {lista_de_identificadores};
```

Es frecuente utilizar enumeraciones con nombre, cuya declaración adopta la forma siguiente:

```
enum [nombre_de_tipo] {lista_de_identificadores};
```

Los identificadores que forman parte de enumeraciones no se admiten por teclado ni se pueden escribir directamente en pantalla. Ejemplos:

```
#include <stdio.h>

enum mis_datos {sistemas,electronica};

enum mis_datos uno, dos;

int main(int argc, const char * argv[]) {
    uno = sistemas;
    dos = electronica;

    printf("%d \n", uno);
    printf("%d \n", dos);
    return 0;
}

#include <stdio.h>

enum mis_datos {sistemas,electronica};
enum mis_datos uno, dos;

int main(int argc, const char * argv[]) {

    enum dias{ lunes, martes, miercoles, jueves, viernes, sabado, domingo};
```

```

    return 0;
}

```

La utilidad de las enumeraciones estriba en que es más fácil recordar un nombre que un determinado valor entero, como en el caso de las constantes creadas mediante `#define`, y además el compilador se encarga de asignar valores.

Las enumeraciones al igual que las estructuras o registros, son una herramienta que le permite al programador un manejo más sencillo de datos.

```

#include <stdio.h>
#include <iostream>
int main(int argc, const char * argv[]) {
    enum autos{chevy, toyota, ferrari, bmw} auto1, auto2, auto3, auto4;

    auto1 = chevy;
    auto2 = toyota;
    auto3 = ferrari;
    auto4 = bmw;
    printf("%d",auto1);
    return 0;
}

```

Es usual encontrar a las enumeraciones como herramienta para trabajar menús. De esta forma se asigna a cada identificador (que son opciones para el usuario) un número entero. Cabe mencionar que las enumeraciones admiten como máximo 256 elementos.

```

#include <stdio.h>
#include <stdlib.h>
#define NUMOPC 5

int main(int argc, const char * argv[]) {
    enum {Nuevo, Abrir, Guardar, Cerrar, Salir};
    char menu[][10] = {"Nuevo", "Abrir", "Guardar", "Cerrar", "Salir"};
    int opc;
}

```

```
do{
    printf("Seleccione la opcion deseada \n");
    for(int i=0; i<NUMOPC; i++){
        printf("%s (%d) ", menu[i], i);
    }
    scanf("%d",&opc);
    switch(opc){
        case Nuevo : printf("Seleccionaste la opcion %s \n",menu[opc]);
                        break;
        case Abrir : printf("Seleccionaste la opcion %s \n",menu[opc]);
                        break;
        case Guardar : printf("Seleccionaste la opcion %s \n",menu[opc]);
                        break;
        case Cerrar : printf("Seleccionaste la opcion %s \n",menu[opc]);
                        break;
        case Salir : printf("Seleccionaste la opcion %s \n",menu[opc]);
                        break;
        default : printf("Opción incorrecta \n");
                  break;
    }//switch
}while(opc != Salir);
return 0;
}
```